

LABORATORY OBSERVATION / RECORD

Course	: Full Stack Web Development
Course Code	: 23CM4121
Assignment No.	: 2
Student Name	: EMANDI HARI CHARAN
Roll No.	: A24126552080
Date	: 06 / 10 / 2026

1. TITLE

Student Record Management System using MongoDB

2. AIM

To build a Student Record Management System using MongoDB that demonstrates database creation, CRUD operations (Create, Read, Update, Delete), sorting, indexing, and query performance analysis using the MongoDB Shell (mongosh).

3. OBJECTIVE

- Create a MongoDB database (collegeDB) and a collection (students).
- Insert at least 8 student records with fields: rollNo, name, branch, year, marks, email.
- Retrieve all records and filter records based on branch, marks, and roll number.
- Update specific fields (marks, email) of a student using `updateOne()` and `$set`.
- Delete a student record using `deleteOne()`.
- Sort student records by marks in descending order using `sort()`.
- Create an index on `rollNo` and demonstrate its performance benefit using `explain()`.

4. THEORY

MongoDB is a NoSQL, document-oriented database that stores data as BSON (Binary JSON) documents inside collections. Unlike relational databases, it does not require a fixed schema, making it flexible for rapidly changing data structures.

- **Database:** A container for collections (e.g., collegeDB).
- **Collection:** A group of documents, similar to a table in SQL (e.g., students).
- **Document:** A JSON-like record stored inside a collection (e.g., one student's data).
- **CRUD Operations:** Create (`insertMany()`), Read (`find()`), Update (`updateOne` + `$set`), Delete (`deleteOne`).
- **Comparison Operators:** `$gt` (greater than), `$gte` ($\geq$), `$lt` (less than), `$lte` ($\leq$).
- **Indexing:** An index is a B-Tree data structure that allows MongoDB to locate documents in $O(\log n)$ time instead of scanning every document (COLLSCAN $\rightarrow$ IXSCAN).
- **`explain("executionStats")`:** Shows the query execution plan — stage (IXSCAN/COLLSCAN), documents examined, keys examined, and execution time.

5. ALGORITHM / PROCEDURE

1. Open the terminal and start MongoDB Shell by running: `mongosh`
2. Switch to / create the database: `db = db.getSiblingDB("collegeDB")`
3. Drop the existing collection (for fresh start) and create `students` collection.

4. Insert 8 student records using `db.students.insertMany([...])`.
5. Retrieve all records using `db.students.find()`.
6. Filter by branch using `db.students.find({ branch: "CSE-AIML" })`.
7. Filter by marks using comparison operators: `{ marks: { $gt: 75 } }`.
8. Search by roll number: `db.students.find({ rollNo: "23CM001" })`.
9. Apply multiple conditions (AND): `{ year: 3, marks: { $gte: 80 } }`.
10. Update marks using `updateOne()` with `$set` operator.
11. Update email field of another student using `updateOne()` and `$set`.
12. Delete a student record using `deleteOne({ rollNo: "23CS088" })`.
13. Sort records by marks in descending order: `.sort({ marks: -1 })`.
14. Create an index on rollNo: `db.students.createIndex({ rollNo: 1 })`.
15. Analyse query performance using `.explain("executionStats")` — verify IXSCAN stage.
16. Run extension queries: marks > 80, marks < 50, highest scorer, CSE branch, sorted list.
17. Execute the script: `mongosh --file "assignment 2.js"`

6. PROGRAM

assignment 2.js

```
// Assignment 2: Student Record Management System using MongoDB
// Database: collegeDB | Collection: students

// HOW TO RUN:
// mongosh → load("assignment 2.js")
// OR paste directly into MongoDB Compass (_mongosh tab)

// STEP 1: Switch to / create the database
db = db.getSiblingDB("collegeDB");
print("Switched to database: " + db.getName());

// STEP 2: Create the collection (drop first to start fresh)
db.students.drop();
db.createCollection("students");
print("\n[SUCCESS] Collection 'students' created.");

// STEP 3: Insert 8 student records
print("\n--- Step 3: Inserting Student Records ---");

db.students.insertMany([
  { rollNo: "23CM001", name: "Ravi Kumar",    branch: "CSE-AIML", year: 3, marks: 85, email: "ravi@example.com" },
  { rollNo: "23CM002", name: "Priya Sharma",  branch: "CSE-AIML", year: 3, marks: 92, email: "priya@example.com" },
  { rollNo: "23CS015", name: "Arun Varma",    branch: "CSE",      year: 2, marks: 78, email: "arun@example.com" },
  { rollNo: "23IT040", name: "Sneha Reddy",   branch: "IT",       year: 3, marks: 68, email: "sneha@example.com" },
  { rollNo: "23EC012", name: "Karthik Nair",   branch: "ECE",      year: 1, marks: 45, email: "karthik@example.com" },
  { rollNo: "23CM033", name: "Ananya Roy",    branch: "CSE-AIML", year: 3, marks: 88, email: "ananya@example.com" },
  { rollNo: "23CS088", name: "Vikram Singh",  branch: "CSE",      year: 4, marks: 42, email: "vikram@example.com" },
  { rollNo: "23IT021", name: "Divya Patel",   branch: "IT",       year: 2, marks: 95, email: "divya@example.com" }
]);

print("[SUCCESS] Inserted 8 student documents.");

// STEP 4: Display all students
print("\n--- Step 4: All Students ---");
db.students.find().forEach(printjson);
```

```

// STEP 5: Students in a specific branch
print("\n--- Step 5: Students in 'CSE-AIML' Branch ---");
db.students.find({ branch: "CSE-AIML" }).forEach(printjson);

// STEP 6: Students with marks > 75
// $gt = greater than
print("\n--- Step 6: Students with Marks > 75 ---");
db.students.find({ marks: { $gt: 75 } }).forEach(printjson);

// STEP 7: Search by rollNo
print("\n--- Step 7: Search Student '23CM001' ---");
db.students.find({ rollNo: "23CM001" }).forEach(printjson);

// STEP 8: Multiple conditions -- year = 3 AND marks ≥ 80
// $gte = greater than or equal
print("\n--- Step 8: Year 3 Students with Marks ≥ 80 ---");
db.students.find({ year: 3, marks: { $gte: 80 } }).forEach(printjson);

// STEP 9: Update marks using $set (only updates specified field)
print("\n--- Step 9: Update Marks of '23CM001' to 90 ---");
db.students.updateOne(
  { rollNo: "23CM001" },
  { $set: { marks: 90 } }
);
db.students.find({ rollNo: "23CM001" }).forEach(printjson);

// STEP 10: Update email field
print("\n--- Step 10: Update Email of '23CM002' ---");
db.students.updateOne(
  { rollNo: "23CM002" },
  { $set: { email: "priya.aihl@college.edu" } }
);
db.students.find({ rollNo: "23CM002" }).forEach(printjson);

// STEP 11: Delete a student record
print("\n--- Step 11: Delete Student '23CS088' ---");
db.students.deleteOne({ rollNo: "23CS088" });
print("Records remaining: " + db.students.countDocuments());

// STEP 12: Sort by marks descending (-1 = highest first)
print("\n--- Step 12: Students Sorted by Marks (Highest First) ---");
db.students.find().sort({ marks: -1 }).forEach(printjson);

// STEP 13: Create an index on rollNo for faster searches
print("\n--- Step 13: Create Index on rollNo ---");
db.students.createIndex({ rollNo: 1 });
db.students.getIndexes().forEach(printjson);

// STEP 14: Show query performance -- IXSCAN (with index) vs COLLSCAN (without)
print("\n--- Step 14: Query Performance with Index ---");
const stats = db.students.find({ rollNo: "23CM001" }).explain("executionStats");
print("Stage          : " + stats.executionStats.executionStages.stage);
print("Docs Examined   : " + stats.executionStats.totalDocsExamined);

```

```

print("Keys Examined   : " + stats.executionStats.totalKeysExamined);
print("Execution Time  : " + stats.executionStats.executionTimeMillis + " ms");

// REAL-TIME EXTENSION QUERIES

// 1. Students scoring above 80
print("\n--- Extension 1: Marks > 80 ---");
db.students.find({ marks: { $gt: 80 } }).forEach(printjson);

// 2. Students scoring below 50
print("\n--- Extension 2: Marks < 50 ---");
db.students.find({ marks: { $lt: 50 } }).forEach(printjson);

// 3. Highest scoring student -- sort desc, take first result
print("\n--- Extension 3: Highest Scoring Student ---");
db.students.find().sort({ marks: -1 }).limit(1).forEach(printjson);

// 4. Students in CSE branch
print("\n--- Extension 4: CSE Branch Students ---");
db.students.find({ branch: "CSE" }).forEach(printjson);

// 5. All students sorted by marks
print("\n--- Extension 5: All Students Sorted by Marks ---");
db.students.find().sort({ marks: -1 }).forEach(printjson);

print("\n=====");
print("All Operations Completed!");
print("=====\\n");

```

7. CODE EXPLANATION

Syntax / Command	Explanation
<code>db.getSiblingDB("collegeDB")</code>	Selects (or creates) the database named collegeDB.
<code>db.students.drop()</code>	Drops (deletes) the students collection if it already exists, ensuring a clean start.
<code>db.createCollection("students")</code>	Explicitly creates a new empty collection called students.
<code>db.students.insertMany([...])</code>	Inserts multiple documents (student records) into the collection in a single call.
<code>db.students.find()</code>	Retrieves and returns all documents from the students collection.
<code>db.students.find({ branch: "CSE-AIML" })</code>	Filters documents where the branch field exactly matches "CSE-AIML".
<code>{ marks: { \$gt: 75 } }</code>	<code>\$gt</code> is the "greater than" comparison operator. Matches documents where marks > 75.
<code>{ marks: { \$gte: 80 } }</code>	<code>\$gte</code> is the "greater than or equal to" operator. Matches marks ≥ 80.
<code>{ marks: { \$lt: 50 } }</code>	<code>\$lt</code> is the "less than" operator. Matches documents where marks < 50.
<code>db.students.updateOne(filter, { \$set: { ... } })</code>	Updates ONE matching document. <code>\$set</code> modifies only the specified fields without replacing the entire document.
<code>db.students.deleteOne({ rollNo: "23CS088" })</code>	Permanently removes the first document where rollNo matches "23CS088".

Syntax / Command	Explanation
<code>.sort({ marks: -1 })</code>	Sorts the result set in descending order of marks (-1 = descending, 1 = ascending).
<code>.limit(1)</code>	Restricts the result to only the first document — used with <code>sort()</code> to get the top scorer.
<code>db.students.createIndex({ rollNo: 1 })</code>	Creates an ascending B-Tree index on the <code>rollNo</code> field for faster lookups.
<code>.explain("executionStats")</code>	Returns detailed query execution info: stage (IXSCAN/COLLSCAN), docs examined, keys examined, and time taken.
<code>db.students.countDocuments()</code>	Returns the total number of documents currently in the <code>students</code> collection.

8. TEST CASES AND EXECUTION DATA

Test Case	Operation	Input / Query	Expected Result	Actual Result	Status
TC-01	Insert records	<code>insertMany([8 docs])</code>	8 documents inserted	8 documents inserted successfully	Pass
TC-02	Display all	<code>find()</code>	All 8 records returned	All 8 records displayed	Pass
TC-03	Filter by branch	<code>find({ branch: "CSE-AIML" })</code>	3 students (Ravi, Priya, Ananya)	3 records returned correctly	Pass
TC-04	Filter marks > 75	<code>find({ marks: { \$gt: 75 } })</code>	5 students	5 records returned correctly	Pass
TC-05	Search by rollNo	<code>find({ rollNo: "23CM001" })</code>	Ravi Kumar's record	Ravi Kumar returned correctly	Pass
TC-06	Multi-condition filter	<code>find({ year:3, marks:{\$gte:80} })</code>	3 students (Ravi, Priya, Ananya)	3 records returned correctly	Pass
TC-07	Update marks	<code>updateOne({rollNo:"23CM001"}, {\$set:{marks:90}})</code>	marks updated to 90	marks = 90 confirmed	Pass
TC-08	Update email	<code>updateOne({rollNo:"23CM002"}, {\$set:{email: ... }})</code>	email updated	email = priya.aiml@college.edu confirmed	Pass
TC-09	Delete record	<code>deleteOne({ rollNo: "23CS088" })</code>	Record deleted, 7 remain	Records remaining: 7 ✓	Pass
TC-10	Sort descending	<code>find().sort({ marks: -1 })</code>	Highest: Divya (95), Lowest: Karthik (45)	Correct descending order	Pass
TC-11	Create index	<code>createIndex({ rollNo: 1 })</code>	Index <i>rollNo_1</i> created	Index <code>rollNo_1</code> listed in <code>getIndexes()</code>	Pass
TC-12	Index performance	<code>find({rollNo:"23CM001"}).explain()</code>	Stage: FETCH/IXSCAN,	Stage: FETCH, Keys: 1, Docs: 1, Time: 1ms	Pass

Test Case	Operation	Input / Query	Expected Result	Actual Result	Status
			Docs Examined: 1		

9. OUTPUT

Output 1: Execution of MongoDB Queries in mongosh Terminal

```

PS C:\Users\Hari Charan\OneDrive\Desktop\fullstack\assignments\assignment-2> mongosh --file "assignment 2.js"
Switched to database: collegeDB

[SUCCESS] Collection 'students' created.

--- Step 3: Inserting Student Records ---
[SUCCESS] Inserted 8 student documents.

--- Step 4: All Students ---
{ rollNo: '23CM001', name: 'Ravi Kumar', branch: 'CSE-AIML', year: 3, marks: 85 }
{ rollNo: '23CM002', name: 'Priya Sharma', branch: 'CSE-AIML', year: 3, marks: 92 }
{ rollNo: '23CS015', name: 'Arun Varma', branch: 'CSE', year: 2, marks: 78 }
{ rollNo: '23IT040', name: 'Sneha Reddy', branch: 'IT', year: 3, marks: 68 }
{ rollNo: '23EC012', name: 'Karthik Nair', branch: 'ECE', year: 1, marks: 45 }
{ rollNo: '23CM033', name: 'Ananya Roy', branch: 'CSE-AIML', year: 3, marks: 88 }
{ rollNo: '23IT021', name: 'Divya Patel', branch: 'IT', year: 2, marks: 95 }

--- Step 9: Update Marks of '23CM001' to 90 ---
{ rollNo: '23CM001', name: 'Ravi Kumar', marks: 90 }

--- Step 11: Delete Student '23CS088' ---
Records remaining: 7

--- Step 12: Sorted by Marks (Highest First) ---
{ rollNo: '23IT021', name: 'Divya Patel', marks: 95 }
{ rollNo: '23CM002', name: 'Priya Sharma', marks: 92 }
{ rollNo: '23CM001', name: 'Ravi Kumar', marks: 90 }
{ rollNo: '23CM033', name: 'Ananya Roy', marks: 88 }
{ rollNo: '23CS015', name: 'Arun Varma', marks: 78 }

--- Step 14: Query Performance with Index ---
Stage      : FETCH
Docs Examined : 1
Keys Examined  : 1
Execution Time : 1 ms

=====
All Operations Completed!
=====

```

10. RESULT

The Student Record Management System was successfully implemented using MongoDB Shell (mongosh). All 14 steps were executed without errors — including database and collection creation, insertion of 8 student records, retrieval with filters and comparison operators, updateOne and deleteOne operations, descending sort, index creation on rollNo, and query performance verification via explain("executionStats"). The index scan (IXSCAN/FETCH) confirmed that MongoDB used the created index to locate documents efficiently with only 1 key examined and 1 ms execution time. All extension queries also returned the expected results. The program executed successfully and all 12 test cases passed.